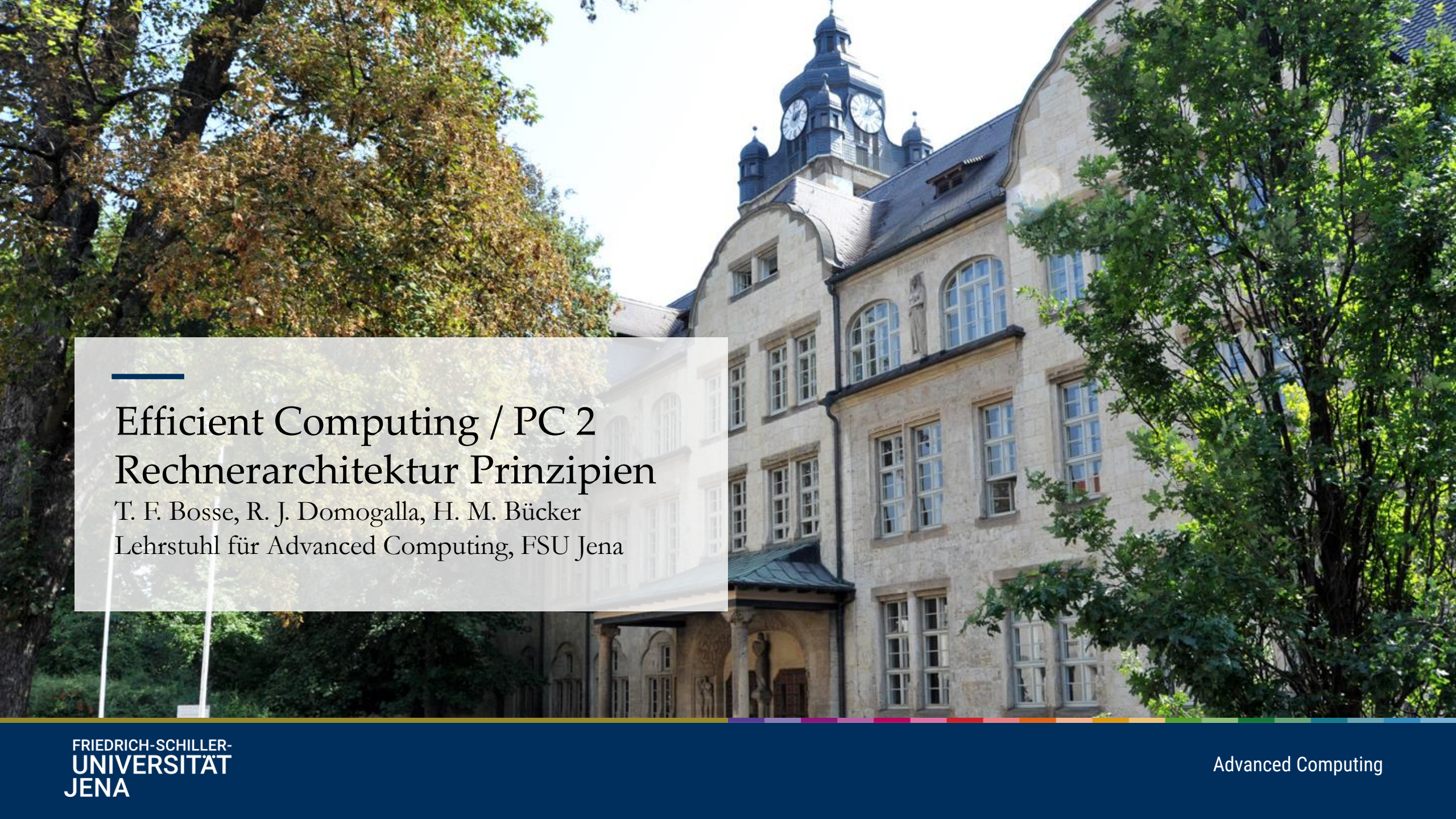


The background of the slide is a photograph of a historic building, likely the Friedrich-Schiller-Universität Jena, featuring a prominent clock tower with two faces. The building is surrounded by lush green trees, and the scene is captured in bright daylight. A semi-transparent white box is overlaid on the left side of the image, containing the title and authors.

Efficient Computing / PC 2

Rechnerarchitektur Prinzipien

T. F. Bosse, R. J. Domogalla, H. M. Bucker
Lehrstuhl für Advanced Computing, FSU Jena

Rechtlicher Hinweis

Diese Vorlesung ist urheberrechtlich geschützt und darf nur im Rahmen von Lehrveranstaltungen der Friedrich-Schiller-Universität Jena verwendet werden. Eine Nutzung durch Verbreitung oder Veröffentlichung dieses Materials — auch in Auszügen — ist strengstens untersagt und wird die Geltendmachung von Unterlassungs- und Schadenersatzansprüchen durch die Friedrich-Schiller-Universität Jena zur Folge haben.

Entwicklung auf Software-Seite

- Auf der Software Seite existiert eine Fülle von div. Anwendungssystemen: *Textverarbeitung, Zahlenverarbeitung, Datenbanken, Multimedia, Spiele, Internet, ...*
- Die verschiedenen Anwendungssysteme haben oft unterschiedlichen Anforderungen: *Zuverlässigkeit, Wartbarkeit, Sicherheit, Portabilität, Leistung, ...*
- Software-Entwicklungszeiten oft vergleichsweise langsam
- Hohe Flexibilität erwünscht
- Hoher Bedarf an neuen Funktionalitäten
- Wunsch nach hoher Performance

Insgesamt eher langsame Entwicklung mit hoher Heterogenität in Anforderungen

Entwicklung auf Hardware-Seite

Auf der Hardware Seite sollen/müssen Realisierungen gewisse Randbedingungen erfüllen:

- zwingend hohe Performance
- zwingend geringer Preis
- Wunsch nach Einfachheit bzw. Regularität
- Neue Technologien möglichst schnell verfügbar machen
- Produktion hoher Stückzahlen

Insgesamt eher schnelle zeitliche Entwicklung mit großer Homogenität in Technologie !

Instruction Architecture Set (ISA)

Computer führt Befehle aus, die Daten bewegen oder transformieren
ISA ist eine Spezifikation einer Menge von Befehlen, die

- eine Hardware korrekt implementieren muss.
- eine effiziente Kontrolle von „Prozessor“ (Embedded System, Cloud Server, ...), Speicher (Register, Cache, RAM, ...) und I/O Geräten (GPIO, FPGA, Bildschirm, Touchpad, Mikrofon, Bluetooth, Netzwerk, externer Speicher,...) erlaubt.
- gewünschte Zielfunktionen (schnelle Befehlsausführung, preisgünstig, Energie effizient) optimiert.

Liste bekannter ISA

ISA	Supporters	Application Areas
x86	AMD, Intel	laptops, PCs, servers
ARM	Apple, ARM, MediaTek, Qualcomm, Samsung	smart devices, smartphones, IoT
RISC-V	RISC-V International	driven by collaborative community

ISA = Schnittstelle zwischen Hard- und Software,
die Portabilität und Korrektheit garantiert !

RISC-V Greencard

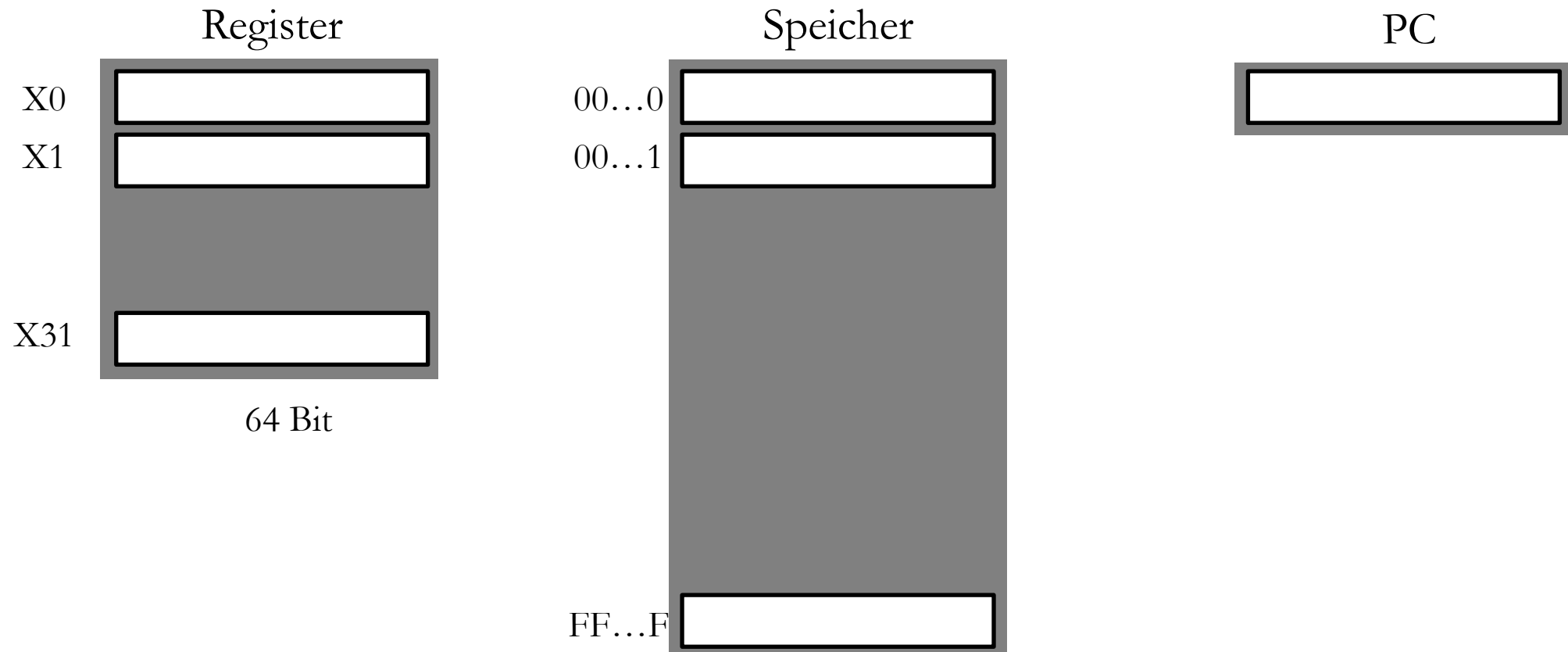
Open Reference Card ①

Base Integer Instructions: RV32I and RV64I					RV Privileged Instructions							
Category	Name	Fmt	RV32I Base		+RV64I		Category	Name	Fmt	RV mnemonic		
Shifts	Shift Left Logical	R	SLL	rd,rs1,rs2	SLLW	rd,rs1,rs2	Trap	Mach-mode trap return	R	MRET		
	Shift Left Log. Imm.	I	SLLI	rd,rs1,shamt	SLLIW	rd,rs1,shamt		Supervisor-mode trap return	R	SRET		
	Shift Right Logical	R	SRL	rd,rs1,rs2	SRLW	rd,rs1,rs2	Interrupt	Wait for Interrupt	R	WFI		
	Shift Right Log. Imm.	I	SRLI	rd,rs1,shamt	SRLIW	rd,rs1,shamt		MMU Virtual Memory FENCE	R	SFENCE.VMA rs1,rs2		
	Shift Right Arithmetic	R	SRA	rd,rs1,rs2	SRAW	rd,rs1,rs2	Examples of the 60 RV Pseudoinstructions					
	Shift Right Arith. Imm.	I	SRAI	rd,rs1,shamt	SRAIW	rd,rs1,shamt						
Arithmetic	ADD	R	ADD	rd,rs1,rs2	ADDW	rd,rs1,rs2	Branch = 0 (BEQ rs,x0,imm)				J	BEQZ rs,imm
	ADD Immediate	I	ADDI	rd,rs1,imm	ADDIW	rd,rs1,imm	Jump (uses JAL x0,imm)				J	J imm
	SUBtract	R	SUB	rd,rs1,rs2	SUBW	rd,rs1,rs2	MoVe (uses ADDI rd,rs,0)				R	MV rd,rs
	Load Upper Imm	U	LUI	rd,imm	Optional Compressed (16-bit) Instruction Extension: RV32C							
	Add Upper Imm to PC	U	AUIPC	rd,imm								
Logical	XOR	R	XOR	rd,rs1,rs2	Loads	Load Word	CL	C.LW	rd',rs1',imm	LW	rd',rs1',imm*4	
	XOR Immediate	I	XORI	rd,rs1,imm		Load Word SP	CI	C.LWSP	rd,imm	LW	rd,sp,imm*4	
	OR	R	OR	rd,rs1,rs2		Float Load Word SP	CL	C.FLW	rd',rs1',imm	FLW	rd',rs1',imm*8	
	OR Immediate	I	ORI	rd,rs1,imm		Float Load Word	CI	C.FLWSP	rd,imm	FLW	rd,sp,imm*8	

<http://www.riscvbook.com/>

Schematischer Speicheraufbau

Prozessor besteht aus (kleiner) festen Anzahl von Registern und (großem) Byte-adressierbaren Speicher sowie Programm Counter (PC)!



Assembler Befehle für Prozessoren

Ähnlich zu dem folgenden Beispiel für einen arithmetischen Befehl, gibt es Befehle für Kontrollfluss, Sprungbefehle, Datenbewegung,...

`add r3, r1, r2` bedeutet $r3 \leftarrow r1 + r2$

Sequenz von Befehlen

Basiselemente aus höheren Programmiersprachen werden in Assembler-Code bestehend aus einer Reihe von Assembler-Befehlen übersetzt:

C-Code

```
int foo(int x, int y, int z)
int res = (((x+y)-z)+x)+y;
```



Compiler

Assemblercode mit
Assemblerbefehlen

```
// x in x2, y in x3, z in x4
foo:
add x5, x2, x3      # t = x+y
sub x6, x5, x4      # r = t-z
add x6, x6, x2      # r = r+x
add x6, x6, x3      # r = r+y
```

} $n = 4$ Assembler-Befehle

Assembler Befehlsausführung

Befehle sind als 32-Bit (oder 64 Bit) Wort codiert und im Speicher abgelegt.

Die Ausführung eines Assembler-Befehls besteht aus mehreren Teilen:

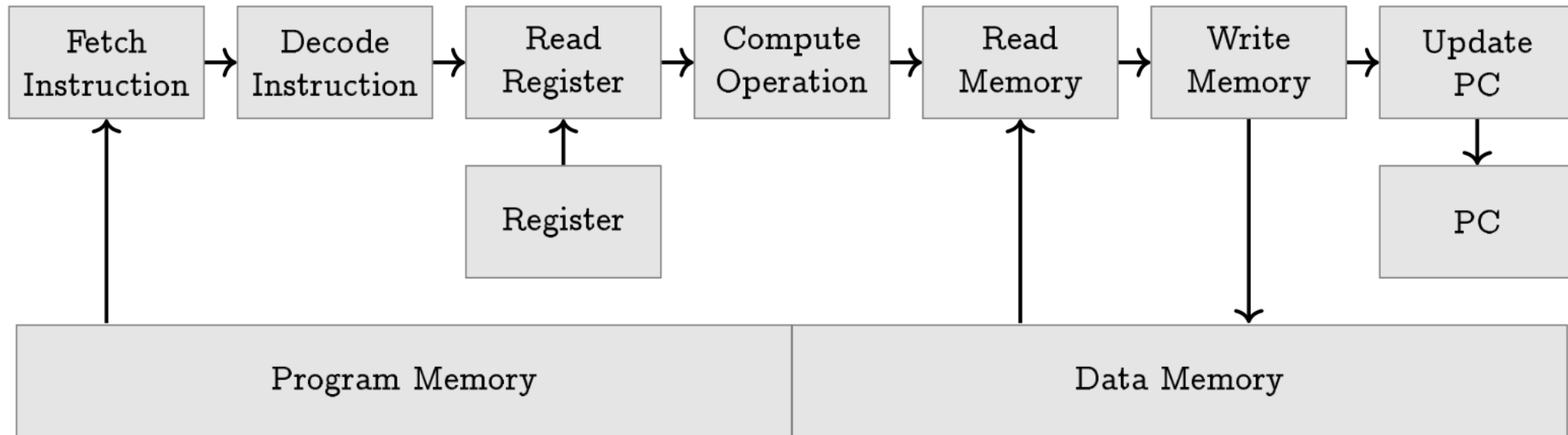
1. Befehl aus Speicher laden (Fetch)
2. Befehlstyp feststellen (Decode)
3. Lade Operanden für Befehl (Read)
4. Führe Operation mit Operanden aus (Compute operation)
5. Lade von Speicher, falls erforderlich (Read)
6. Schreibe Ergebnis in Speicher (Write)
7. Setze den Befehlszähler (Update PC)

Assembler Befehlsausführung

- Ein einzelner Assembler-Befehl ändert (nur) einen kleinen Teil des Prozessorzustands.
- Ein längeres Programm induziert eine geordnete Reihenfolge von Befehlen.
- Sequentielle Abstraktion ist wichtig für Komposition, Modularität, Schlussfolgerung von Ausführung und Korrektheit.
- Diese sequentielle Ausführung in Hardware will/wollte man daher für die Software übertragen.

Hardware für Teilaufgaben

Für jede der Teilaufgaben gibt es separate Hardware Komponenten.



© H.M. Bucker

Die Funktionalität der ISA hat daher eine hohe Bedeutung für die Hardware, insbesondere wenn hohe Performance erwünscht ist.

Möglichkeiten der Beschleunigung

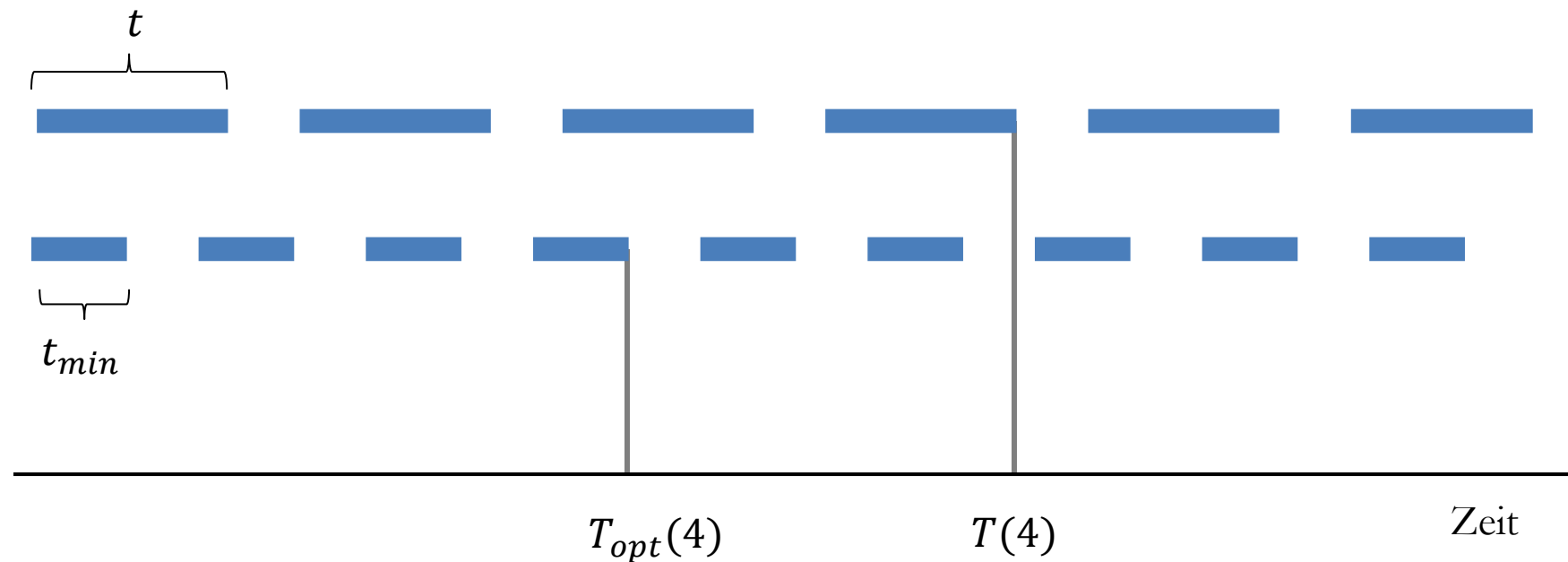
- a. Minimiere die nötige Zeit zur Ausführung der Teilaufgaben.
- b. Instruction-level parallelism:

Idee: Gebe Illusion einer sequentiellen Abstraktion an Software,
aber führe Parallelität auf Hardware Ebene aus.

- i. Pipelining
- ii. Multiple Issue
- iii. Out-of-order Execution
- iv. Speculative Execution

Minimierung der Zeit

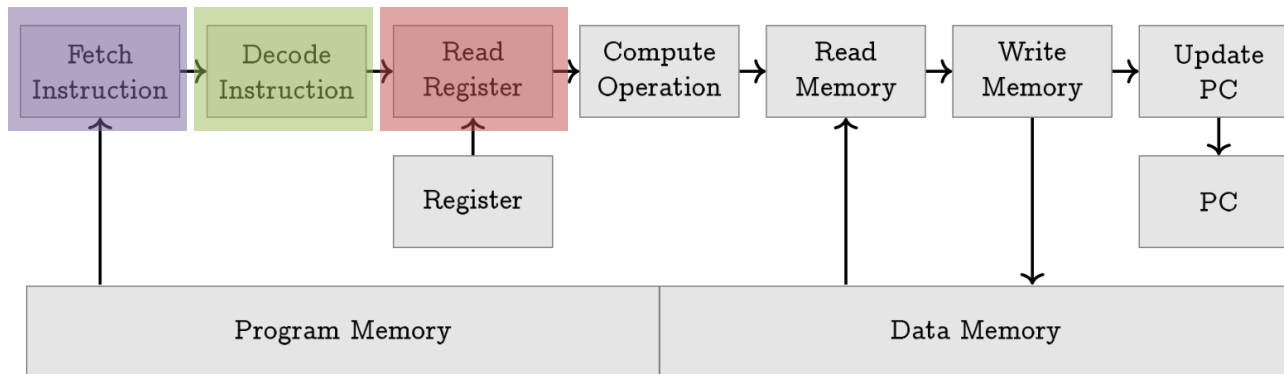
Optimiere die nötige Gesamtzeit $T(n)$ zur Ausführung von n Teilaufgaben durch Minimierung der nötigen Zeit jeder einzelnen Teilaufgabe unter der Annahme, dass alle Teilaufgaben die gleiche Zeit t benötigen.



Dieser Ansatz verletzt nicht den Ansatz der sequentiellen Abstraktion.

Pipelining

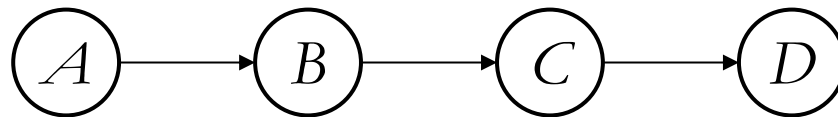
Führe unabhängige Teilaufgaben von unterschiedlichen Instanzen gleichzeitig aus.



```
// x in x2, y in x3, z in x4
foo:
add x5, x2, x3      # t = x+y
sub x6, x5, x4      # r = t-z
add x6, x6, x2      # r = r+x
add x6, x6, x3      # r = r+y
```

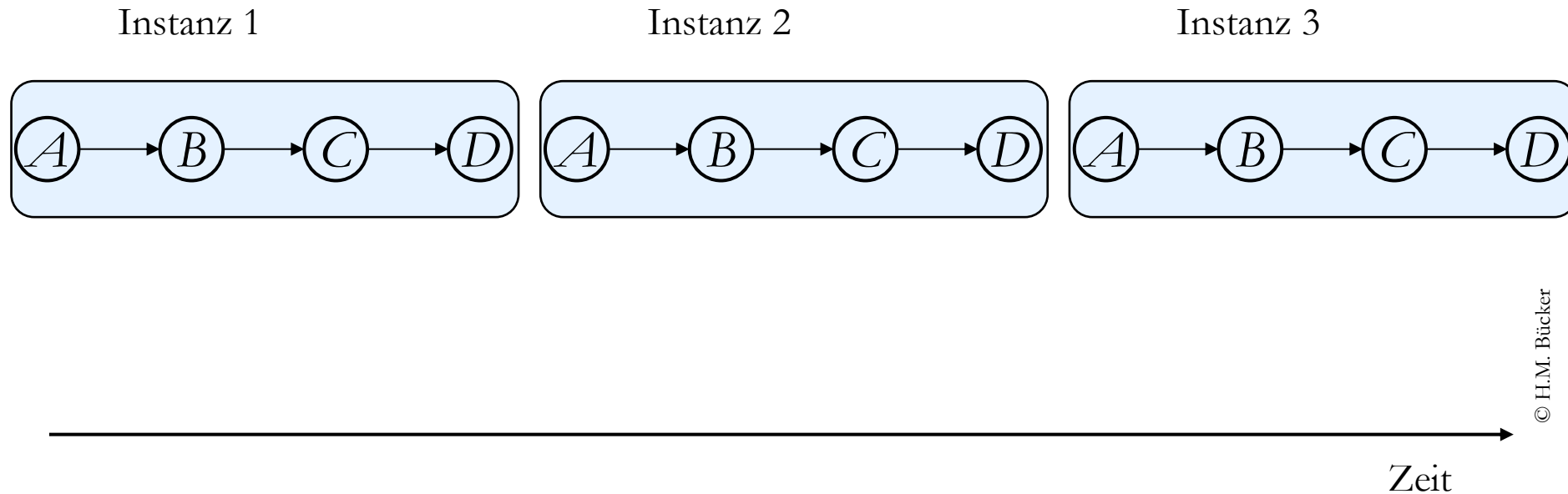
Pipelining Model

- Sei $G = (V, E)$ ein gerichteter Graph.
- Knoten $v \in V$ repräsentiert Teilaufgabe.
- Kante $u \rightarrow v \in E$ repräsentiert Abhängigkeit: Teilaufgabe u muss beendet sein, bevor Teilaufgabe v begonnen werden kann.



Ausführung ohne Pipelining

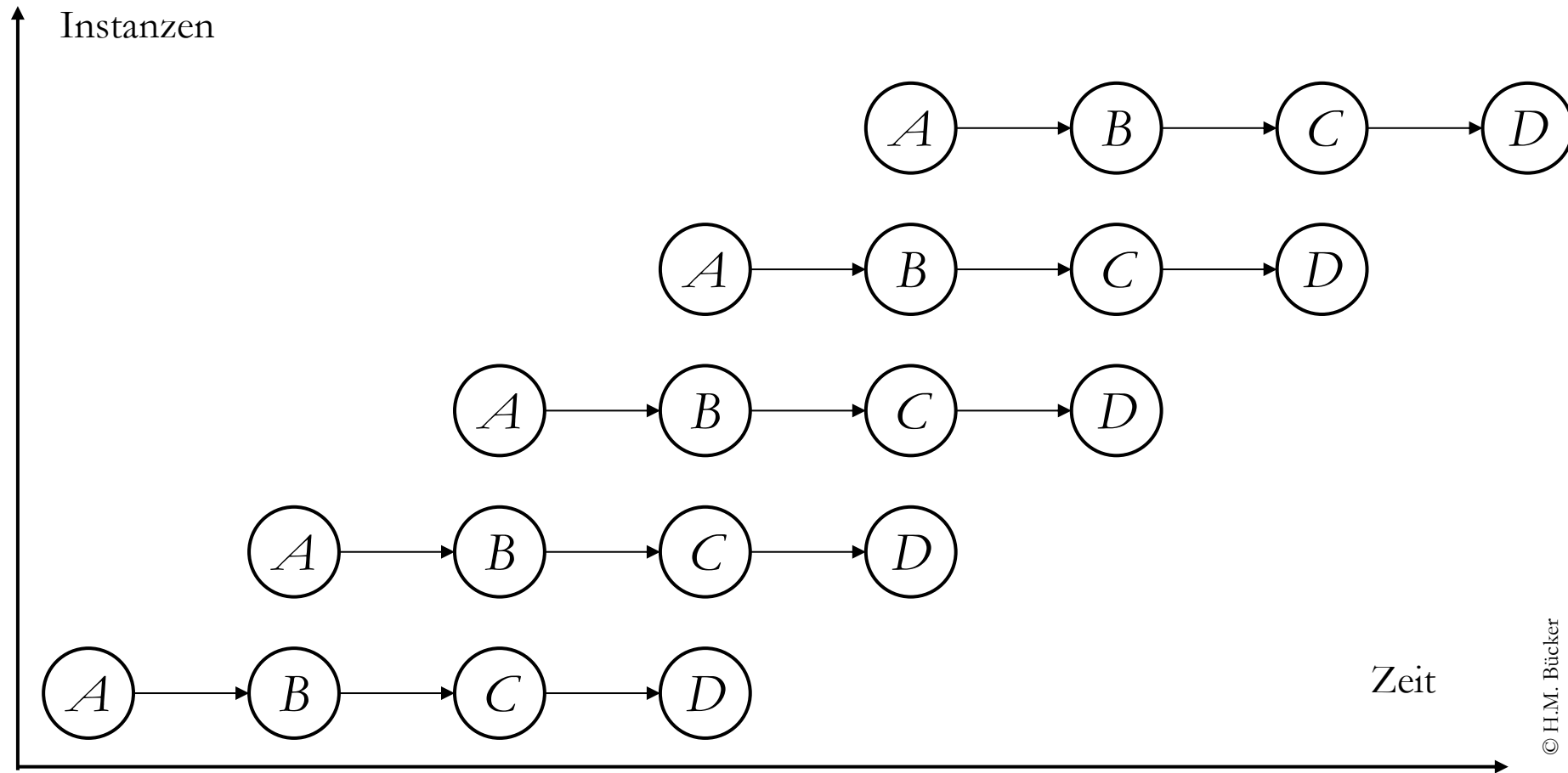
Beispiel von 3 Instanzen mit jeweils 4 Teilaufgaben.



Pipelining

- n Instanzen eines sequentiellen G mit k Teilaufgaben müssen abgearbeitet werden.
- Alle Teilaufgaben benötigen die gleiche Zeit t zur Ausführung.
- Unterschiedliche Teilaufgaben von unterschiedlichen Instanzen können parallel ausgeführt werden.
- Die gleiche Teilaufgabe von unterschiedlichen Instanzen können nicht parallel ausgeführt werden.

Pipelining: $n = 5$ and $k = 4$



Pipelining

- Zeit für n Instanzen ohne Pipelining:
- Zeit für n Instanzen mit Pipelining für Tiefe k :
- Beschleunigung/Speed-up:

Pipelining

- Zeit für n Instanzen ohne Pipelining:

$$T(n, k, t) = nkt$$

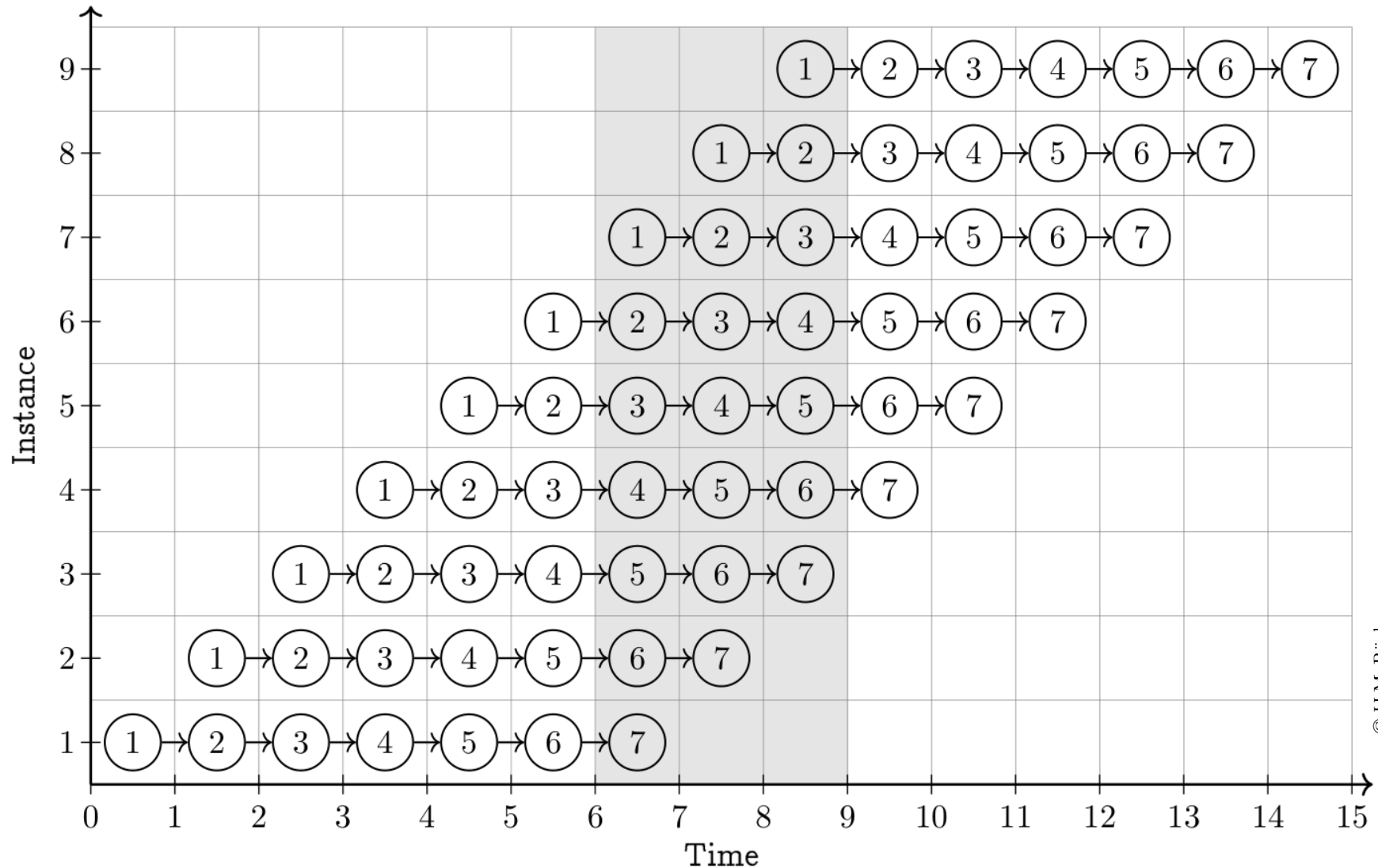
- Zeit für n Instanzen mit Pipelining für Tiefe k :

$$T_{Pipe}(n, k, t) = kt + (n - 1)t$$

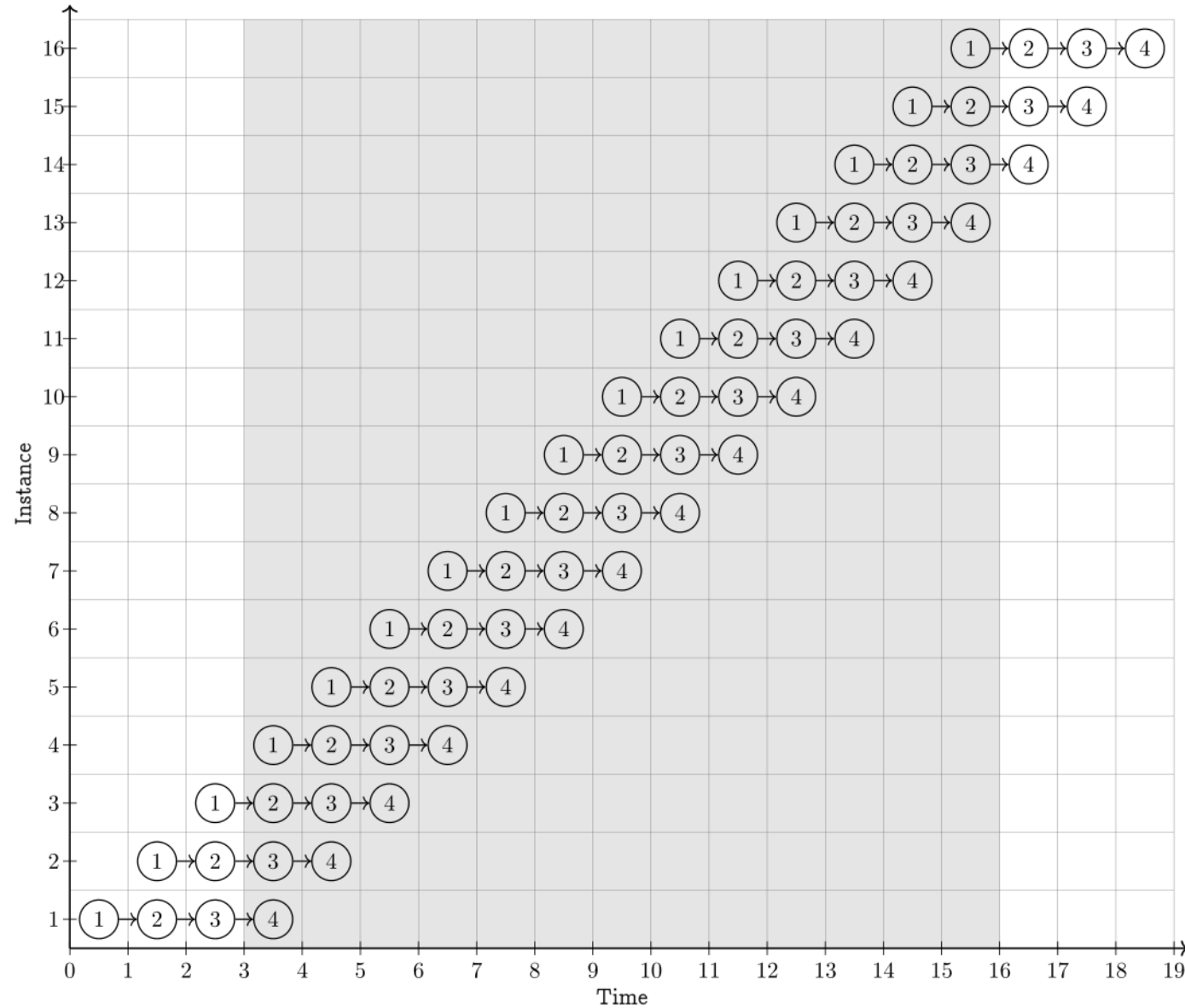
- Beschleunigung/Speed-up:

$$S = \frac{T(n, k, t)}{T_{Pipe}(n, k, t)} = \frac{nkt}{(n + k - 1)t} = \frac{k}{1 + \frac{k - 1}{n}} \xrightarrow{n \rightarrow \infty} k$$

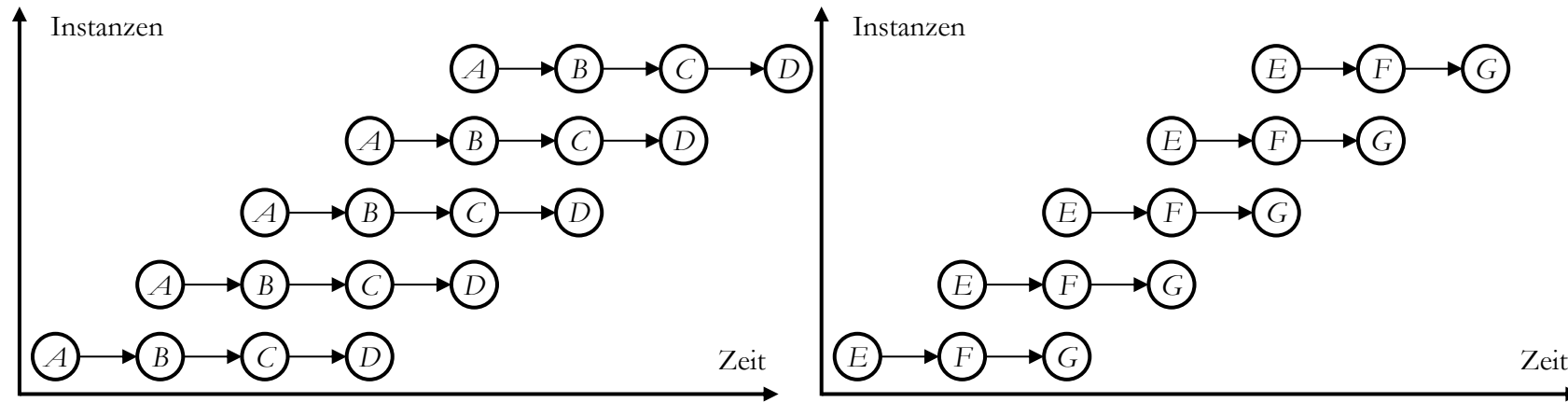
Pipelining with $n = 9$ and $k = 7$



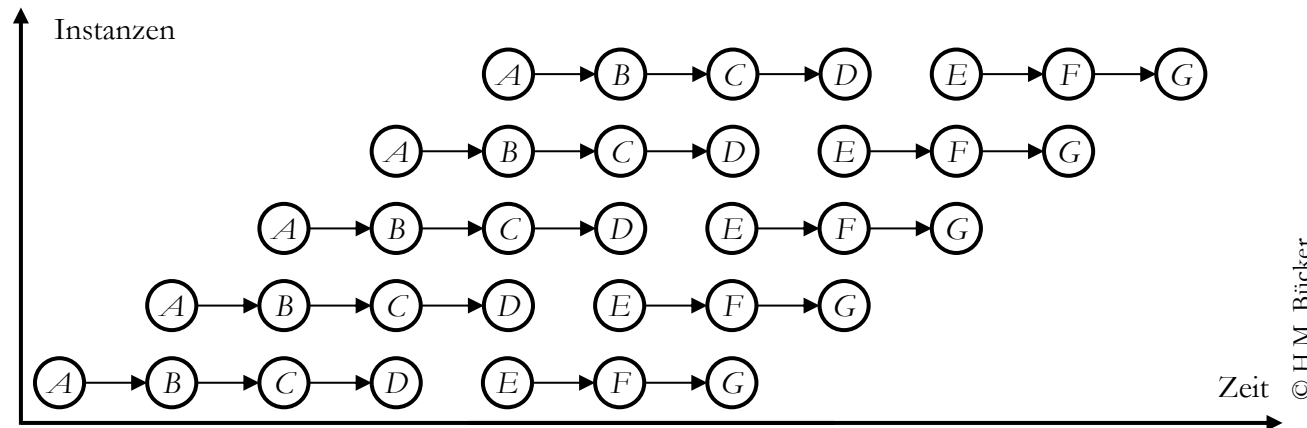
Pipelining with $n = 16$ and $k = 4$



Pipelining



© H.M. Bucker



© H.M. Bucker

Multiple Issue

Falls es Befehle gibt, die unabhängig voneinander ausgeführt werden können, führe diese parallel aus.
(Etwa wenn unterschiedliche Register und Speicherplätze und genügend Hardwarekomponenten vorhanden sind)



SIMD

Single Instruction Multiple Data

Idee: Ersetze Sequenz von identischen Befehlen auf unterschiedlichen skalaren Daten ohne Datenabhängigkeit durch einen Vektorbefehl.

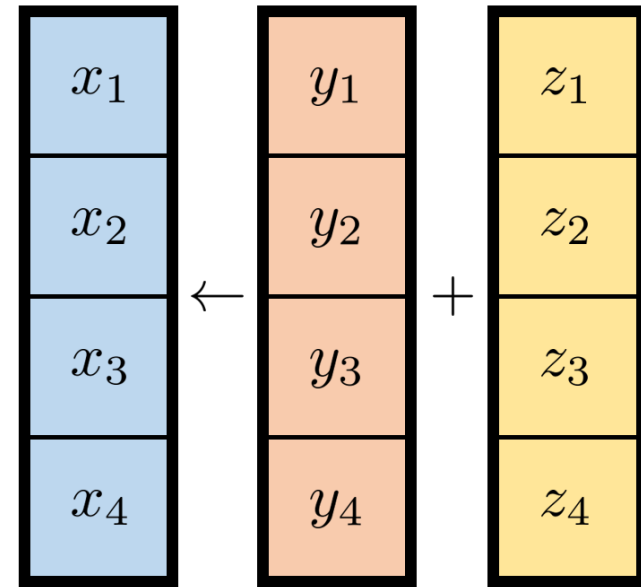
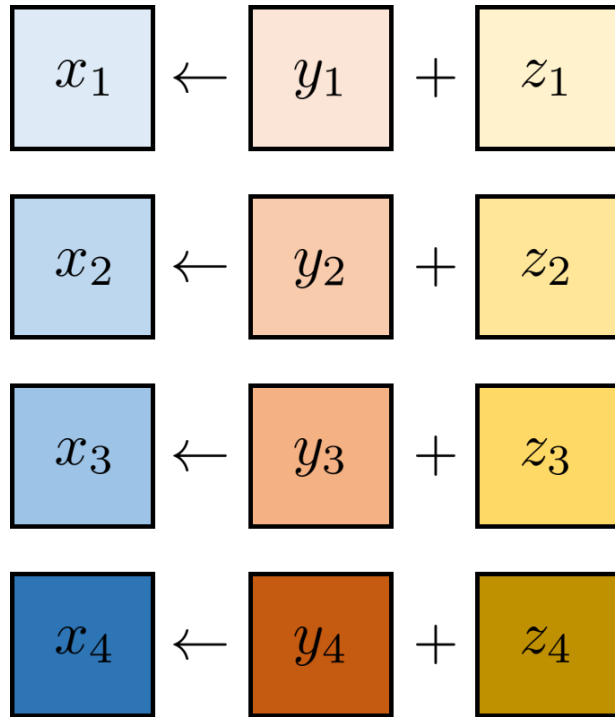
```
foo_orig:  
x1 ← ADD(y1, z1)  
x2 ← ADD(y2, z2)  
x3 ← ADD(y3, z3)  
x4 ← ADD(y4, z4)
```



```
foo_simd:  
x[1:4] ← VECADD(y[1:4], z[1:4])
```

SIMD

Single Instruction Multiple Data



SIMD

Prozessor führt gleiche Befehle auf mehreren Daten aus.

Beispiel: $y = y + \alpha x$ mit $y, x \in \mathbb{R}^n$ und $\alpha \in \mathbb{R}$.

```
foo_orig:
s0 = Load  $\alpha$ 
for i = 1,2,...,n do
    s1 = Load x[i]
    s1 = Mult(s0, s1)
    s2 = Load y[i]
    s2 = Add(s2, s1)
    y[i] = Store(s2)
end
```



```
foo_simd:
s0 = Load  $\alpha$ 
v1 = vecLoad x[:]
v2 = vecMult(s0, v1)
v3 = vecLoad y[:]
v4 = vecAdd(v3, v2)
y = vecStore(v4)
```

Entwicklung von SIMD

Name	Größe	Jahr	Daten Typ	Register
MMX	64-Bit	1997	Packed Integer	mm0-mm7
SSE	128-Bit	1999	Single-precision FP	xmm0-xmm7
SSE2	128-Bit	2000	Double-precision FP, Int	xmm0-xmm15
SSE3/4	128-Bit	2004-2006	Vectorized Operations	xmm0-xmm15
AVX	128-Bit	2011	3-Operand, Packed FP	xmm0-xmm15
AVX2	256-Bit	2013	Integer Extensions	ymm0-ymm15
AVX-512	512-Bit	2017	Large Vector/Masking	zmm0-zmm31